

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#)

[Meus cursos](#)

[PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#)

[Tópico 4](#)

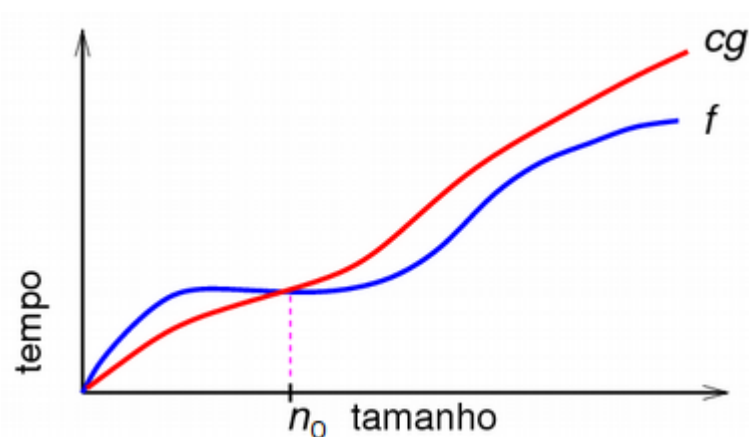
[Notas de aula: notação assintótica](#)

Permite exprimir mais facilmente a complexidade de um algoritmo/problema, pois omite constantes multiplicativas e termos que não dominam a expressão.

Veremos as notações O , Ω , Θ , o , e ω .

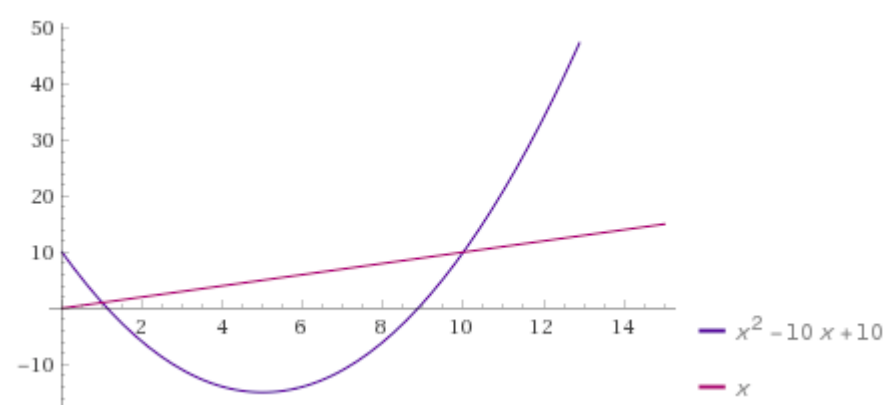
$O(g(n))$ é o conjunto de toda função $f(n)$ tq existem constantes positivas c e n_0 que tornam $0 \leq f(n) \leq cg(n)$ para todo $n \geq n_0$. Ou seja, se $f(n) \in O(g(n))$, então $f(n)$ cresce no máximo tão rapidamente quanto $g(n)$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \text{ vale } 0 \text{ ou cte positiva.}$$



Ex.: $f(n) = n$, $g(n) = n^2 - 10n + 10$. Como $g(n)$ domina $f(n)$, temos que $f(n) \in O(g(n))$.
Porém, só podemos garantir que $f(n) \leq g(n)$ para $n \geq 10$.
Portanto, escolhemos as constantes $c = 1$ e $n_0 = 10$.

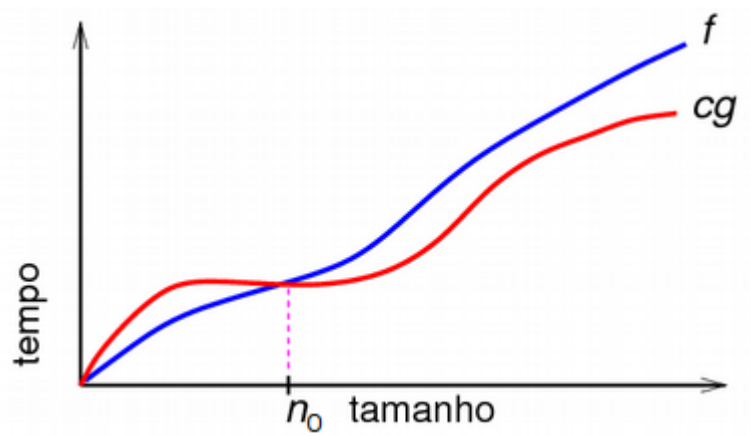
wolframalpha.com: plot $x^2 - 10x + 10$, $1 \cdot x$, $x=0$ to 15



Ex.: $f(n) = 2n + 1$, $g(n) = n$. Como $f(n)$ e $g(n)$ crescem iguais, temos que $f(n) \in O(g(n))$.
Porém, $f(n) > g(n)$ para todo n positivo.
Neste caso basta multiplicar $g(n)$ por uma constante maior que 2. Multiplicando $g(n)$ por 3 temos que $f(n) \leq g(n)$ para todo $n \geq 1$.
Portanto, escolhemos as constantes $c = 3$ e $n_0 = 1$.

$\Omega(g(n))$ é o conjunto de toda função $f(n)$ tq existem constantes positivas c e n_0 que tornam $0 \leq cg(n) \leq f(n)$ para todo $n \geq n_0$. Ou seja, se $f(n) \in \Omega(g(n))$, então $f(n)$ cresce no mínimo tão rapidamente quanto $g(n)$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \text{ vale } \infty \text{ ou cte positiva.}$$



Exemplos anteriores: $n^2 - 10n + 10 \in \Omega(n)$, pois n^2 domina n .

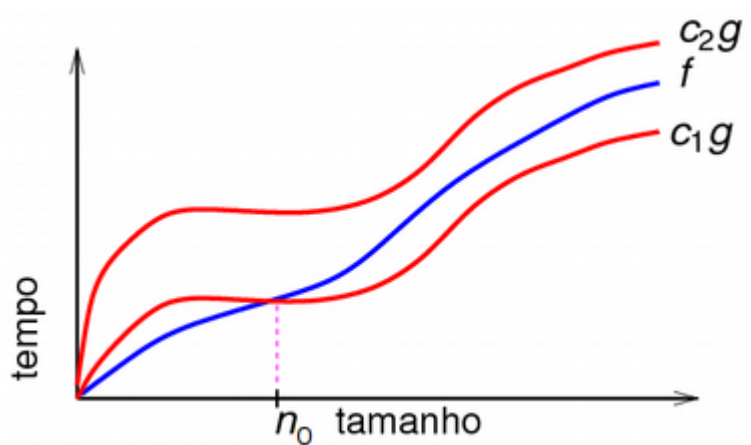
$2n + 1 \in \Omega(n)$, pois crescem iguais.

Ex.: É demonstrado que nenhum algoritmo de ordenação tem tempo de execução que cresce mais lento que $n \log n$; seja, se um algoritmo de ordenação tem tempo $T(n)$, então sabemos que $T(n) \in \Omega(n \log n)$.

Notações O e o ocorrem na análise de pior caso de algoritmos, enquanto Ω e ω geralmente indicam a dificuldade de problemas.

$\Theta(g(n))$ é o conjunto de toda função $f(n)$ tq existem ctes pos. c_1, c_2 e n_0 que tornam $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ todo $n \geq n_0$. Ou seja, se $f(n) \in \Theta(g(n))$, então $f(n)$ cresce tão rapidamente quanto $g(n)$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \text{constante positiva.}$$



Exemplos anteriores: $n^2 - 10n + 10 \notin \Theta(n)$, pois n^2 domina n .

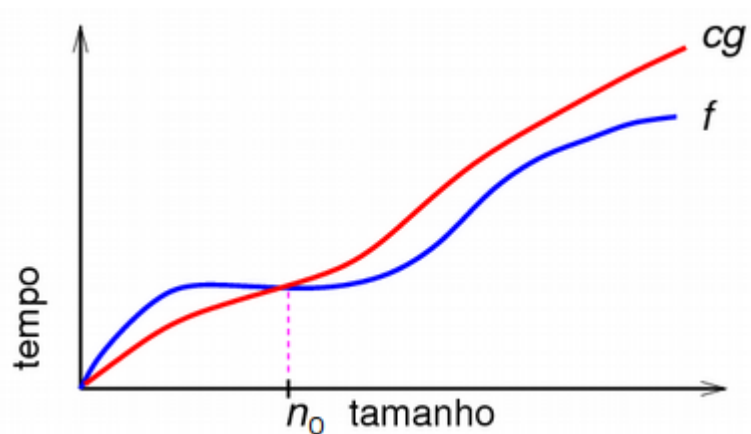
$2n + 1 \in \Theta(n)$, pois crescem iguais.

Alternativamente, após verificar que $2n + 1 \in O(n)$ e $2n + 1 \in \Omega(n)$, podemos concluir que $2n + 1 \in \Theta(n)$.

A notação Θ é preferível na análise de complexidade, pois descreve precisamente o crescimento. Porém, nem sem pena o esforço desta precisão, e nos conformamos com a notação O .

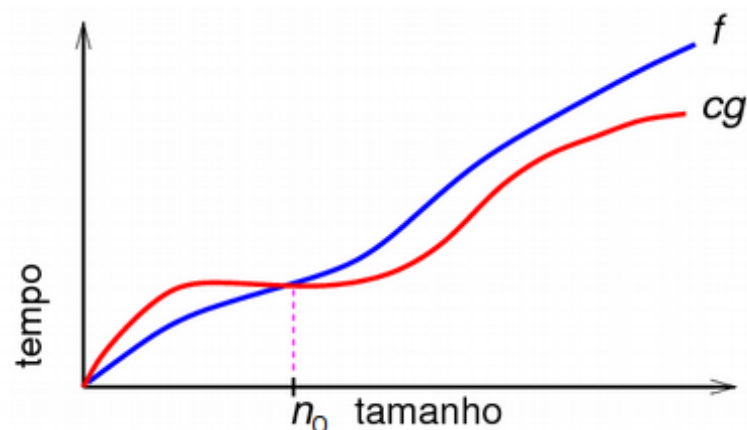
$o(g(n))$ é o conjunto das funções $f(n)$ tq, PARA TODA cte pos. c , existe cte pos. n_0 que torna $0 \leq f(n) < cg(n)$ para $n \geq n_0$. Ou seja, se $f(n) \in o(g(n))$, então $f(n)$ cresce ESTRITAMENTE mais lento que $g(n)$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$



$\omega(g(n))$ é o conjunto das funções $f(n)$ tq, PARA TODA cte pos. c , existe cte pos. n_0 que torna $0 \leq cg(n) < f(n)$ par $n \geq n_0$. Ou seja, se $f(n) \in \omega(g(n))$, então $f(n)$ cresce ESTRITAMENTE mais rápido que $g(n)$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty.$$



Exemplos anteriores: $n^2 - 10n + 10 \notin o(n)$.

$n^2 - 10n + 10 \in \omega(n)$.

$2n + 1 \notin o(n)$.

$2n + 1 \notin \omega(n)$.

Ex.: $n \in o(n^2)$? Para todo $c > 0$ precisamos achar um n_0 tq $f(n) < cg(n)$ p/ $n > n_0$. Ou seja, $n < cn^2$ para todo n

Como n é positivo, divida os dois lados por n , obtendo $1 < cn$.

Ou seja, $n > 1/c$.

Portanto, para cada valor de $c > 0$, basta escolher para n_0 um valor maior que $1/c$. Por exemplo, $n_0 = 1 + 1/c$.

Ex.: $n^2 \in o(n)$? Para todo $c > 0$ precisamos achar um n_0 tq $f(n) < cg(n)$ p/ $n > n_0$. Ou seja, $n^2 < cn$ para todo n

Como n é positivo, divida os dois lados por n , obtendo $n < c$.

Não é possível satisfazer $n < c$ para todo n maior que uma constante n_0 .

Propriedades

Transitividade: Se $f(n) \in O(g(n))$ e $g(n) \in O(h(n))$, então $f(n) \in O(h(n))$.

Se $f(n) \in \Omega(g(n))$ e $g(n) \in \Omega(h(n))$, então $f(n) \in \Omega(h(n))$.

Se $f(n) \in \Theta(g(n))$ e $g(n) \in \Theta(h(n))$, então $f(n) \in \Theta(h(n))$.

Se $f(n) \in o(g(n))$ e $g(n) \in o(h(n))$, então $f(n) \in o(h(n))$.

Se $f(n) \in \omega(g(n))$ e $g(n) \in \omega(h(n))$, então $f(n) \in \omega(h(n))$.

Para o caso da notação o :

Assuma $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$ e $\lim_{n \rightarrow \infty} g(n)/h(n) = 0$. Então,

$$\lim_{n \rightarrow \infty} \frac{f(n)}{h(n)} = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \cdot \frac{g(n)}{h(n)} = 0.$$

Reflexividade: $f(n) \in O(f(n))$.

$f(n) \in \Omega(f(n))$.

$f(n) \in \Theta(f(n))$.

$$\lim_{n \rightarrow \infty} \frac{f(n)}{f(n)} = 1 \text{ (constante positiva).}$$

Simetria: $f(n) \in \Theta(g(n))$ sse $g(n) \in \Theta(f(n))$.

Se c é uma constante positiva,

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \Leftrightarrow \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \frac{1}{c} \text{ (cte positiva).}$$

Simetria transposta: $f(n) \in O(g(n))$ sse $g(n) \in \Omega(f(n))$.

$f(n) \in o(g(n))$ sse $g(n) \in \omega(f(n))$.

$$0 \leq \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty \iff 0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} \leq \infty.$$

Algoritmos eficientes

Convencionou-se que um algoritmo é chamado **eficiente** quando seu tempo de execução é limitado por um polinô seja, pertence a $O(n^k)$ para alguma constante $k > 0$.

Ex.: n^5 , $\log n$, $(\log n)^3$, $n \log n$.

Ex.: funções não limitadas por polinômios: n^n , 2^n , $n!$.

Por que polinômios?

- Aumentar n por um fator constante gera um aumento de fator constante no tempo de execução.

Ex.: $T(n) = 4n^3$. Dobrando a entrada o tempo será $4(2n)^3 = 4 \cdot 2^3 n^3$ (8x maior). Para $T(n) = 2^n$, dobrar a entrada resulta em tempo $2^{2n} = 2^n \cdot 2^n$ (2^n vezes maior).
- Funciona bem na prática: embora n^{100} cresça muito rápido, quanto $T(n)$ é polinomial geralmente não tem grau. Por outro lado, problemas em que não se conhece algoritmo polinomial geralmente consomem tempo inviável.
- Facilidade de manipulação teórica: produto e soma de polinômios resulta em polinômio.

Ex.: Algoritmo para calcular n -ésimo número de Fibonacci.

$$F_n = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ F_{n-1} + F_{n-2} & \text{se } n > 1 \end{cases}$$

n	0	1	2	3	4	5	6	7	8	
fib	0	1	1	2	3	5	8	13	21	..
fib(n):										
se n = 0, retorne 0										
se n = 1, retorne 1										
retorne fib(n-1) + fib(n-2)										

Complexidade: $T(0) = 2, T(1) = 3, T(n) = T(n - 1) + T(n - 2) + 3 > F_n$ para $n \geq 2$.

Fácil provar por indução que $F_n \geq 2^{n/2}$ p/ $n \geq 6$. Logo, $T(n) \in \Omega(1,4^n)$.

Provar por indução que $F_n \geq 2^{n/2}$ para $n \geq 6$.

Caso base: $F_6 = 8 \geq 2^{6/2} = 8, F_7 = 13 > 2^{7/2} (\approx 11.4)$.

Para $n \geq 8$, assumo que vale para $n - 1$ e $n - 2$.

$$\begin{aligned} F_n &= F_{n-1} + F_{n-2} \geq 2^{(n-1)/2} + 2^{(n-2)/2} = 2^{n/2-1/2} + 2^{n/2-1} \\ &= 2^{n/2} (2^{-1/2} + 2^{-1}) > 1.2 \cdot 2^{n/2} > 2^{n/2}. \end{aligned}$$

Ex.: Algoritmo para calcular n -ésimo número de Fibonacci. Armazenando os dois últimos números de Fibonacci (prog. dinâmica):

fib(n):	
se n = 0, retorne 0	1
a <- 0, b <- 1, f <- 1	3
para i <- 2 até n	n
a <- b, b <- f, f <- a + b	3(n-1)
retorne f	1

$$T(n) = 5 + n + 3(n - 1) = 4n + 2 = \Theta(n)$$

Na verdade é **pseudo-polinomial**, pois o valor de n é armazenado em $\log n$ bits. Ou seja, se recebe b bits na entrada $T(n) = \Theta(n) = \Theta(2^b)$.

O algoritmo anterior é $\Omega(1,4^{2^b})!$

Ex.: Algoritmo para calcular n -ésimo número de Fibonacci. Utilizando a equação

$$F_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

Neste caso gastamos tempo linear em n para calcular a potenciação quando a multiplicação tem tempo constante (precisão fixa).

Na verdade é possível calcular a^n em tempo $\log n$ por divisão e conquista. Ex.:
 $a^8 = a^4 a^4 = (a^2 a^2)(a^2 a^2) = ((a \cdot a)(a \cdot a))((a \cdot a)(a \cdot a))$ gasta 3 multiplicações. $T(n) = \Theta(b)$.

Se a precisão não for fixa, gastamos $O(b^2)$ para multiplicar.
Na verdade é possível multiplicar em $O(b^{1.59})$ com divisão e conquista.

◀ [Notas de aula: comparação do crescimento de funções](#)

Seguir para...

[Vídeo: demonstração de propriedades da notação Theta \(opcional\) \(7min\)](#) ▶

©2020 - Universidade Federal do Ceará - Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro - Quixadá - Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

 [Baixar o aplicativo móvel.](#)